

Question 1

A data engineering team is building a Silver events table from a Bronze events table using Lakeflow Spark Declarative Pipelines.

New event records continuously arrive in the Bronze table throughout the day. The team wants the Silver table to:

- process newly arriving records incrementally
- remain queryable by downstream analytics teams
- be managed by Lakeflow Spark Declarative Pipelines
- integrate with Unity Catalog governance

Which type of table best meets these requirements?

- A.** Managed table
- B.** External table
- C.** Streaming table
- D.** Materialized view

Correct answer: C

C. Streaming table

Explanation

Streaming tables are designed for incremental processing workloads where new records continuously arrive over time.

In Lakeflow Spark Declarative Pipelines, streaming tables automatically process newly arriving data from upstream streaming sources while remaining queryable like standard tables.

Streaming tables are commonly used in Bronze and Silver layers where pipelines continuously ingest and refine incoming data.

Materialized views are more commonly used in Gold layers for serving precomputed aggregations and analytics results to downstream reporting workloads.

Question 2

A data engineering team runs several workloads in Databricks with usage patterns that change throughout the day. Some workloads need compute resources quickly during peak periods, but there are also long idle periods when no processing is running.

The team wants to reduce operational overhead by avoiding manual cluster sizing, startup management, and idle compute management.

Which compute option is most appropriate?

- A. Serverless compute
- B. All-purpose compute
- C. Job compute
- D. Single-node compute

Correct answer: A

A. Serverless compute

Explanation

The key requirement is to reduce compute management overhead while supporting workloads with changing demand. Serverless compute is the best fit because it removes much of the manual responsibility for provisioning, sizing, starting, scaling, and stopping compute resources.

This makes it suitable when teams want Databricks to handle more of the infrastructure lifecycle automatically.

Question 3

A global retail company needs to continuously ingest changes from a SQL Server operational database into Unity Catalog governed Delta tables in Databricks.

The ingestion solution must:

- capture inserts, updates, and deletes incrementally
- avoid repeatedly reloading unchanged data
- minimize custom ingestion code
- support reliable enterprise ingestion at scale

Which approach best meets these requirements?

- A.** Use nightly JDBC ingestion jobs
- B.** Use COPY INTO with scheduled database exports
- C.** Use Auto Loader with JDBC ingestion from the database
- D.** Use Lakeflow Connect database connector

Correct answer: D

D. Use Lakeflow Connect database connector

Explanation

The scenario focuses on continuously ingesting incremental changes from an operational relational database into Databricks.

A managed database ingestion solution should:

- support incremental ingestion patterns
- avoid loading unchanged data
- minimize custom operational logic
- provide scalable enterprise ingestion capabilities

Lakeflow Connect database connectors are designed for these types of enterprise database ingestion workloads while integrating with Unity Catalog governed Delta tables.

Question 4

A manufacturing company ingests IoT machine telemetry data into a Bronze table using PySpark.

Each record contains:

- a machine_id
- an event_timestamp
- an array column named sensor_readings

For example:

```
{
  "machine_id": "M101",
  "event_timestamp": "2026-05-10T10:15:00",
  "sensor_readings": [12.5, 13.1, 11.8]
}
```

The analytics team requires one row per sensor reading for downstream aggregation of sensor metrics

Which transformation best meets these requirements?

A.

```
silver_df = bronze_df.select(
    "machine_id",
    "event_timestamp",
    flatten(col("sensor_readings")).alias("sensor_reading")
)
```

B.

```
silver_df = bronze_df.select(
    "machine_id",
    "event_timestamp",
    split(col("sensor_readings"), ",").alias("sensor_reading")
)
```

C.

```
silver_df = bronze_df.select(
    "machine_id",
    "event_timestamp",
    col_to_rows(col("sensor_readings")).alias("sensor_reading")
)
```

D.

```
silver_df = bronze_df.select(
    "machine_id",
    "event_timestamp",
    explode(col("sensor_readings")).alias("sensor_reading")
)
```

Correct answer: D

Explanation

The `explode()` function is used in PySpark to transform array elements into individual rows.

In this scenario, each Bronze table record contains a `sensor_readings` array:

```
{
  "machine_id": "M101",
  "event_timestamp": "2026-05-10T10:15:00",
  "sensor_readings": [12.5, 13.1, 11.8]
}
```

Using:

```
explode(col("sensor_readings"))
```

produces multiple output rows:

```
machine_id event_timestamp sensor_reading
M101 2026-05-10T10:15:00 12.5
M101 2026-05-10T10:15:00 13.1
M101 2026-05-10T10:15:00 11.8
```

This type of transformation is commonly used in Silver-layer pipelines when nested or array-based Bronze data needs to be converted into a more relational structure for downstream analytics, aggregations, filtering, and BI reporting.

Question 5

A data engineering team is configuring a production Lakeflow Job that loads data into Unity Catalog tables every hour.

Initially, the engineering team configured the job to run using a personal user account belonging to one of the engineers.

During a security review, the team identifies the following concerns:

- pipelines may fail if the employee leaves the company
- password changes could interrupt scheduled workloads
- audit tracking for automated workloads should be separated from individual users

Which approach best addresses these concerns?

- A.** Use a service principal for the Lakeflow Job.
- B.** Use a personal access token generated from an engineer's user account.
- C.** Use a shared personal user account for all production Lakeflow Jobs.
- D.** Store engineer credentials securely in Azure Key Vault and use them for the job.

Correct answer: A

- A.** Use a service principal for the Lakeflow Job.

Explanation

Service principals are commonly used for automated Databricks workloads such as:

- Lakeflow Jobs
- ingestion pipelines
- scheduled ETL workloads
- CI/CD deployments

Using a service principal provides several advantages for production environments:

- reduces dependency on individual employees
- improves auditability for automated workloads
- avoids failures caused by password resets or disabled user accounts
- supports centralized permission management

In production systems, automated workloads should generally use dedicated non-human identities rather than personal user accounts or user-generated tokens.

Question 6

A data engineering team stores raw telemetry data in cloud object storage outside the managed storage location of Unity Catalog.

The engineering team wants to create a Unity Catalog table that:

- references the existing data files in cloud storage
- allows the data files to remain accessible even if the table is dropped
- enables multiple external systems to continue using the same storage location

Which approach best meets these requirements?

- A.** Create a temporary view referencing the cloud storage path.
- B.** Create a managed table
- C.** Create a managed Delta table and periodically export the files to external storage.
- D.** Create an external table

Correct answer: D

- D.** Create an external table

Explanation

External tables are commonly used when:

- data already exists in cloud storage
- multiple systems need to access the same files
- storage lifecycle management must remain outside Unity Catalog
- dropping the table should not remove the underlying data files

External tables are created by specifying a `LOCATION` clause that points to an existing storage path.

In contrast, managed tables allow Databricks to manage both metadata and storage lifecycle operations.

Question 7

A data engineering team manages a Unity Catalog table named `finance.gold.customers`.

Analysts query the table directly using the three-level namespace:

```
SELECT customer_id, customer_name, ssn  
FROM finance.gold.customers;
```

The team wants users in the finance department to see the real SSN values, while users outside the finance department should see masked values automatically when querying the same table.

Which Unity Catalog feature best meets this requirement?

- A. Dynamic views
- B. Column masks
- C. Adaptive Query Execution
- D. Row filters

Correct answer: B

B. Column masks

Explanation

Column masks in Unity Catalog allow engineering teams to centrally protect sensitive columns such as:

- Social Security Numbers
- salary information
- credit card numbers
- personally identifiable information (PII)

In this scenario:

- users query the table directly using the three-level namespace
- the engineering team wants masking to occur automatically at query time
- different users should see different values for the same column

Column masks are designed specifically for this type of centralized column-level security enforcement.

Unlike dynamic views, column masks allow governance policies to be enforced directly on the table itself without requiring users to query a separate view.

Question 8

A retail company is building a Silver sales dataset using PySpark.

The sales DataFrame contains:

- all sales transactions
- a small lookup DataFrame containing product category mappings

The engineering team notices that the join operation is causing excessive shuffle activity and increasing execution time.

The lookup DataFrame contains only a few hundred rows and can fit easily in memory.

Which join strategy would best improve the performance of this workload?

- A. Broadcast join**
- B. Cross join
- C. Self join
- D. Left Join

Correct answer: A

A. Broadcast join

Explanation

A broadcast join is an optimization technique used when one side of a join is small enough to fit in executor memory.

Instead of shuffling both DataFrames across the cluster, Spark broadcasts the smaller DataFrame to all executors:

```
broadcast(product_lookup_df)
```

This allows each executor to perform the join locally against its partition of the larger DataFrame.

Broadcast joins are commonly used in Silver and Gold transformation pipelines when joining large transactional datasets with small reference or lookup tables because they help:

- reduce shuffle operations
- lower network traffic
- improve join performance

Question 9

A data engineering team creates the following objects in Unity Catalog:

- Catalog: finance
- Schema: reporting
- Table: monthly_revenue

A data analyst is granted the following privilege:

```
GRANT SELECT
ON SCHEMA finance.reporting
TO analysts;
```

The analyst attempts to query the `finance.reporting.monthly_revenue` table but receives an access error.

Which additional privilege is required for the analyst to successfully query the table?

- A.** `MANAGE` on the finance catalog
- B.** `SELECT` on the `monthly_revenue` table
- C.** `USE CATALOG` on the finance catalog
- D.** `MODIFY` on the `monthly_revenue` table

Correct answer: C

C. `USE CATALOG` on the finance catalog

Explanation

Unity Catalog uses a hierarchical security model.

To successfully query a table, users typically require:

- `USE CATALOG` on the catalog
- `USE SCHEMA` on the schema
- `SELECT` on the table

Privileges granted at the schema level can inherit downward to tables within the schema. However, users must still have permission to access the parent catalog before querying objects inside it.

In this scenario:

- the analyst already inherits SELECT access through the schema-level grant
- but they do not have USE CATALOG on the finance catalog

As a result, the query fails before table-level access is evaluated.

Question 10

A data engineer modifies several notebooks inside a Databricks Git Folder connected to a shared Git repository.

The engineer can see all changes inside the Databricks workspace, but other team members cannot see the updates in the remote repository.

Which action most likely has not yet been completed?

- A. The engineer has not yet pulled the latest changes from the remote repository.
- B. The engineer has not yet switched to the main branch.
- C. The engineer has not yet created a pull request for the changes.
- D. The engineer has not yet committed and pushed the changes to the remote repository.

Correct answer: D

- D. The engineer has not yet committed and pushed the changes to the remote repository.

Explanation

Databricks Git Folders integrate directly with remote Git repositories and support standard Git development workflows.

When working inside a Git Folder:

- notebook changes initially exist only in the local working branch
- changes must be committed to Git history
- committed changes must then be pushed to the remote repository before other team members can access them

A common workflow is:

- Modify notebooks
- Review changes
- Commit changes
- Push changes to the remote repository

- Create a pull request if required

Understanding the distinction between:

- local workspace changes
- committed changes
- pushed remote changes

is important for collaborative CI/CD workflows in Databricks.

Question 11

A retail company maintains a customer dimension table using Lakeflow Spark Declarative Pipelines.

Customer records can change over time, including:

- address updates
- membership tier changes
- phone number updates

The analytics team needs to:

- automatically process CDC updates from the source system
- preserve historical versions of customer records
- track which customer version was active at a given point in time

Which SQL definition best supports these requirements?

A.

```
CREATE OR REFRESH MATERIALIZED VIEW dim_customers AS  
SELECT * FROM customers_bronze_cdc;
```

B.

```
CREATE OR REFRESH STREAMING TABLE dim_customers;
```

```
CREATE FLOW customer_flow AS  
AUTO CDC INTO dim_customers  
FROM stream(customers_bronze_cdc)  
KEYS (customer_id)  
SEQUENCE BY operation_timestamp  
STORED AS SCD TYPE 1;
```

C.

```
CREATE OR REFRESH STREAMING TABLE dim_customers AS  
SELECT DISTINCT * FROM customers_bronze_cdc;
```

D.

```
CREATE OR REFRESH STREAMING TABLE dim_customers;
```

```
CREATE FLOW customer_flow AS  
AUTO CDC INTO dim_customers  
FROM stream(customers_bronze_cdc)  
KEYS (customer_id)  
SEQUENCE BY operation_timestamp  
STORED AS SCD TYPE 2;
```

Correct answer: D

Explanation

SCD Type 2 is used when a dimension table must preserve history instead of only storing the latest version of each record.

In Lakeflow Spark Declarative Pipelines, `AUTO CDC INTO` applies change data from a CDC source into a target streaming table. The `KEYS` clause identifies the business key for matching records, and `SEQUENCE BY` defines the ordering column used to apply changes in the correct sequence.

Using:

```
STORED AS SCD TYPE 2
```

tells the pipeline to maintain historical versions of changed records. This allows analysts to see not only the current customer details, but also previous versions that were valid at earlier points in time.

Question 12

A marketing agency wants to share campaign performance data stored in Databricks with a partner that uses Tableau for analytics but does not have a Databricks workspace.

The partner requires direct access to the shared data from Tableau.

Which type of Delta Sharing should the agency use?

- A. Unity Catalog external locations
- B. Lakehouse Federation
- C. Databricks-to-Databricks protocol
- D. Open Sharing protocol

Correct answer: D

D. Open Sharing protocol

Explanation

Delta Sharing supports:

- Databricks-to-Databricks sharing
- Open Sharing

Databricks-to-Databricks sharing is used when both organizations use Unity Catalog-enabled Databricks workspaces.

Open Sharing is designed for external systems and tools such as:

- Tableau
- Power BI
- pandas
- Apache Spark

allowing secure read-only access to shared Delta tables without requiring the recipient to use Databricks.

Question 13

A data engineering team is building an Auto Loader pipeline to ingest continuously arriving JSON files from cloud object storage into a Bronze Delta table.

The team wants the pipeline to:

- read JSON files incrementally as they arrive
- use Auto Loader rather than a standard file source
- track schema information in a dedicated schema location

Which code snippet best meets these requirements?

A.

```
df = (  
    spark.readStream  
        .format("cloudFiles")  
        .option("cloudFiles.format", "json")  
        .option("cloudFiles.schemaLocation",  
            "/Volumes/formula1/checkpoints/orders_schema")  
        .load("/Volumes/formula1/landing/orders")  
)
```

B.

```
df = (  
    spark.read  
        .format("cloudFiles")  
        .option("cloudFiles.format", "json")  
        .option("cloudFiles.schemaLocation",  
            "/Volumes/formula1/checkpoints/orders_schema")  
        .load("/Volumes/formula1/landing/orders")  
)
```

C.

```
df = (  
    spark.readStream  
        .format("json")  
        .option("cloudFiles.schemaLocation",  
            "/Volumes/formula1/checkpoints/orders_schema")  
        .load("/Volumes/formula1/landing/orders")  
)
```

D.

```
df = (  
    spark.readStream  
        .format("cloudFiles")  
        .option("format", "json")  
        .option("schemaLocation", "/Volumes/formula1/checkpoints/orders_schema")  
        .load("/Volumes/formula1/landing/orders")  
)
```

Correct answer: A

Explanation

Auto Loader is designed for scalable incremental ingestion of files arriving in cloud object storage.

A typical Auto Loader ingestion pipeline uses:

- `spark.readStream`
- `.format("cloudFiles")`
- `cloudFiles.format` to define the underlying file type
- `cloudFiles.schemaLocation` to persist schema tracking information

This allows ingestion pipelines to efficiently process continuously arriving files while maintaining schema tracking across runs.

Question 14

A company receives thousands of JSON files every hour in cloud object storage, and the source folder is expected to grow to millions of files over time.

The ingestion pipeline must:

- detect newly arrived files incrementally
- avoid manual tracking of processed files
- scale efficiently as file volume grows
- land data into Unity Catalog governed Delta tables

Which ingestion approach is most appropriate?

- A. COPY INTO**
- B. Use external tables**
- C. JDBC ingestion**
- D. Auto Loader**

Correct answer: D

D. Auto Loader

Explanation

The scenario focuses on continuously ingesting very large volumes of files from cloud object storage with minimal operational overhead.

Auto Loader is designed specifically for this type of workload and provides:

- scalable incremental ingestion
- automatic file discovery
- efficient handling of directories containing millions of files
- reduced operational management overhead
- direct ingestion into Delta tables

This makes Auto Loader a strong fit for large-scale continuous ingestion pipelines in Databricks.

Question 15

A Lakeflow Job consists of five sequential tasks. The final task generates a critical audit log required for compliance reporting.

The data engineer wants to receive a Slack notification only if the final audit log task fails, without receiving notifications for failures in earlier tasks.

Which configuration best meets this requirement?

- A. Use cluster-level logging and external monitoring tools to notify Slack when the audit log task fails.
- B. Configure Slack notifications for all tasks and filter unwanted alerts inside Slack.
- C. Configure a task-level notification on the audit log task to trigger a Slack alert only if that task fails.
- D. Configure a job-level notification that triggers on task failures and sends alerts to Slack.

Correct answer: C

- C. Configure a task-level notification on the audit log task to trigger a Slack alert only if that task fails.

Explanation

Lakeflow Jobs support both job-level and task-level notifications, allowing engineering teams to configure alerts with different levels of granularity. Task-level notifications are useful when alerts should only be triggered for failures in specific critical tasks rather than for every task failure in the workflow.

Question 16

A financial services company continuously ingests transactions into a Delta table from multiple concurrent pipelines.

The engineering team wants to ensure:

- failed writes never leave partially updated data
- concurrent readers always see consistent results
- committed changes remain durable after failures

Which Delta Lake capability directly provides these guarantees?

- A. Delta Caching
- B. ACID transactions
- C. Liquid Clustering
- D. Predictive Optimization

Correct answer: B

B. ACID transactions

Explanation

The company is processing transactions through multiple concurrent ingestion pipelines, making transactional consistency extremely important.

Delta Lake uses ACID transactions to ensure:

- incomplete writes are not exposed to downstream readers
- concurrent operations do not corrupt table state
- committed updates remain durable after failures

These guarantees are especially important in production environments where multiple streaming and batch workloads may read and write to the same Delta tables simultaneously.

Without ACID transaction support, concurrent updates could lead to inconsistent reads, partially written data, or corrupted table state.

Question 17

A data engineering team is building a Silver customer table from a Bronze dataset using PySpark.

The Bronze table contains a `signup_date` column stored as a string in the format `yyyy-MM-dd`.

The analytics team requires this column to be stored as a `DATE` type in the Silver table for downstream reporting and filtering.

Which transformation best meets this requirement?

A.

```
silver_df = bronze_df.withColumn(  
    "signup_date",  
    col("signup_date").cast("timestamp")  
)
```

B.

```
silver_df = bronze_df.withColumnRenamed(  
    "signup_date",  
    to_date(col("signup_date"), "yyyy-MM-dd")  
)
```

C.

```
silver_df = bronze_df.withColumn(  
    "signup_date",  
    to_date(col("signup_date"), "yyyy-MM-dd")  
)
```

D.

```
silver_df = bronze_df.withColumn(  
    "signup_date",  
    col("signup_date").format("yyyy-MM-dd")  
)
```

Correct answer: C

Explanation

Using `to_date(col("signup_date"), "yyyy-MM-dd")` correctly converts the string column into a Spark SQL `DATE` type using the specified input format.

Question 18

A healthcare organization stores patient records in a Unity Catalog table named `patients`.

The compliance team requires that:

- members of the `medical_staff` group can see the full value of the `diagnosis` column
- all other users should see the value `REDACTED`

Which function correctly implements this requirement?

A.

```
CREATE FUNCTION diagnosis_mask(diagnosis STRING)
RETURN CASE
    WHEN is_account_group_member('medical_staff')
    THEN 'REDACTED'
    ELSE diagnosis
END;
```

B.

```
CREATE FUNCTION diagnosis_mask(diagnosis STRING)
RETURN CASE
    WHEN is_account_group_member('medical_staff')
    THEN diagnosis
    ELSE 'REDACTED'
END;
```

C.

```
CREATE FUNCTION diagnosis_mask(diagnosis STRING)
RETURN CASE
    WHEN is_user_group_member() = 'medical_staff'
    THEN 'REDACTED'
    ELSE diagnosis
END;
```

D.

```
CREATE FUNCTION diagnosis_mask(diagnosis STRING)
RETURN CASE
    WHEN is_user_group_member('medical_staff')
    THEN diagnosis
    ELSE 'REDACTED'
END;
```

Correct answer: B

Explanation

Column masking functions are used to dynamically determine what value should be returned to a user when they query a sensitive column.

In this example, the function uses `is_account_group_member('medical_staff')` to check whether the current user belongs to the `medical_staff` group.

If the user is a member of that group, the function returns the original `diagnosis` value. Otherwise, it returns the masked value `REDACTED`.

This approach allows all users to query the same table while ensuring that only authorized users can view sensitive information in the `diagnosis` column. Users who are not members of the `medical_staff` group still see the row, but the sensitive column value is masked.

Question 19

A data engineering team creates a Unity Catalog table using the following SQL statement:

```
CREATE TABLE finance.gold.monthly_sales  
AS  
SELECT * FROM finance.silver.sales;
```

Later, the team runs:

```
DROP TABLE finance.gold.monthly_sales;
```

Which statement correctly describes what happens to the underlying data files?

- A.** The underlying data files are immediately and permanently deleted from cloud storage when the table is dropped.
- B.** The table is automatically converted into an external table before the underlying data files are removed.
- C.** The table metadata is removed, and the underlying data files are marked for deletion during managed maintenance operations after the recovery retention period.
- D.** Only the table metadata is removed, while the underlying data files remain permanently in cloud storage.

Correct answer: C

C. The table metadata is removed, and the underlying data files are marked for deletion during managed maintenance operations after the recovery retention period.

Explanation

In Unity Catalog, tables created without a `LOCATION` clause are managed tables.

For managed tables, Databricks manages:

- the table metadata
- the storage location
- lifecycle management of the underlying data files

When a managed table is dropped:

- the table metadata is removed
- the table can still be recovered during the recovery retention period using `UNDROP TABLE`

- after the recovery period expires, the underlying data is deleted during Databricks-managed maintenance operations

In contrast, external tables only remove metadata when dropped, while the underlying files remain in cloud storage.

Question 20

A data engineering team is combining historical sales data with newly arrived sales data using PySpark before writing the results into a Silver table.

Both the historical and current sales DataFrames contain the same columns, but the column order is not consistent between the datasets.

Historical DataFrame columns:

```
["customer_id", "product_id", "quantity"]
```

Current DataFrame columns:

```
["product_id", "quantity", "customer_id"]
```

Once combined, the engineering team notices that some values are appearing under incorrect columns in the Silver table.

Which approach best resolves this issue?

- A. `combined_df = historical_df.unionAll(current_df)`
- B. `combined_df = historical_df.unionByColumn(current_df)`
- C. `combined_df = historical_df.unionByName(current_df)`
- D. `combined_df = historical_df.union(current_df)`

Correct answer: C

C. `combined_df = historical_df.unionByName(current_df)`

Explanation

When combining DataFrames in PySpark, standard union operations align columns by position rather than by column name.

If source systems generate datasets with inconsistent column ordering, this can lead to incorrect data being written into downstream Silver tables.

`unionByName()` is commonly used in ingestion and transformation pipelines to safely combine DataFrames when column ordering may vary between datasets.

Question 21

A data engineering team manages multiple Lakeflow Jobs and Lakeflow Spark Declarative Pipelines across development, test, and production environments.

The engineering team wants to:

- standardize deployments across environments
- avoid manually recreating jobs and pipelines in each workspace
- package deployment configurations together with workspace assets
- support CI/CD-based deployment workflows

Which approach best meets these requirements?

- A.** Use Declarative Automation Bundles to package and deploy Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, and related workspace assets across environments.
- B.** Use Databricks Git Folders to synchronize Lakeflow Jobs and SDP pipelines across environments.
- C.** Export DBC files for Lakeflow Jobs and Lakeflow Spark Declarative Pipelines and import them into each environment.
- D.** Develop Lakeflow Jobs and Lakeflow Spark Declarative Pipelines in a common workspace and execute them from each environment.

Correct answer: A

A. Use Declarative Automation Bundles to package and deploy Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, and related workspace assets across environments.

Explanation

Declarative Automation Bundles are used to package, configure, and deploy Databricks workspace assets across environments.

Bundles commonly manage:

- Lakeflow Jobs
- Lakeflow Spark Declarative Pipelines
- notebooks

- deployment configurations
- environment settings

Bundles help teams:

- standardize deployments
- reuse the same codebase across environments
- support CI/CD workflows
- reduce manual deployment effort
- avoid environment drift

This approach improves deployment consistency and operational maintainability across development, test, and production environments.

Question 22

A data engineer is working with a column containing raw JSON strings and wants to infer the schema from one of the JSON records before parsing the data.

Which Spark SQL function should the engineer use?

- A. `infer_json_schema()`
- B. `get_json_schema()`
- C. `json_schema_extract()`
- D. `schema_of_json()`

Correct answer: D

D. `schema_of_json()`

Explanation

In Spark SQL, when working with raw JSON strings, it's often necessary to infer the schema before parsing the data. This is particularly helpful when you don't already know the structure of the JSON in advance or want to dynamically define the schema for use in functions like `from_json()`.

The function that enables this is: `schema_of_json(json_string)`

This function:

- Takes a JSON string as input.
- Returns the inferred schema as a string in `struct<...>` format.
- Is commonly used to dynamically generate the schema for functions like `from_json()` or `from_json(col, schema_of_json(col))`.

Question 23

A healthcare organization has already created the following column masking function:

```
CREATE FUNCTION diagnosis_mask(diagnosis STRING)
RETURN CASE
    WHEN is_account_group_member('medical_staff')
    THEN diagnosis
    ELSE 'REDACTED'
END;
```

The data engineer now wants to apply this mask to the diagnosis column in the patients table.

Which statement should the engineer use?

A.

```
ALTER TABLE patients
ALTER COLUMN diagnosis
SET DEFAULT diagnosis_mask;
```

B.

```
ALTER TABLE patients
MODIFY COLUMN diagnosis
SET MASK diagnosis_mask;
```

C.

```
ALTER TABLE patients
ALTER COLUMN diagnosis
SET MASK diagnosis_mask;
```

D.

```
ALTER TABLE patients
SET MASK diagnosis_mask ON (diagnosis);
```

Correct answer: C

Explanation

After creating a masking function, the function must be attached to a specific column. Unity Catalog does this using the `ALTER TABLE ... ALTER COLUMN ... SET MASK` statement. Once applied, the masking function is evaluated whenever users query the column, allowing authorized users to see the original value while returning a masked value for everyone else.

Question 24

A data engineering team uses the same Declarative Automation Bundle to deploy a Lakeflow Job across development, test, and production environments.

The engineering team wants:

- the same codebase to be reused across all environments
- different catalog names to be used in each environment
- different cluster sizes to be used in production compared to development

Which approach best meets these requirements?

- A.** Use bundle variables and target-specific overrides for environment-specific configuration values.
- B.** Create separate Git repositories for development, test, and production deployments.
- C.** Create separate copies of the bundle configuration for each environment and maintain them independently.
- D.** Store environment-specific values directly inside notebooks and update them before deployment.

Correct answer: A

- A.** Use bundle variables and target-specific overrides for environment-specific configuration values.

Explanation

Declarative Automation Bundles support environment-specific deployment configuration through:

- variables
- target definitions
- target overrides

This allows teams to:

- reuse the same codebase across environments
- customize deployment settings per environment

- avoid maintaining duplicate deployment logic

Common environment-specific configurations include:

- catalog names
- cluster configurations
- schedules
- workspace targets

Using variables and target overrides improves:

- deployment consistency
- CI/CD maintainability
- environment isolation
- configuration management

This approach is commonly used when promoting the same Lakeflow Jobs and SDP pipelines across development, test, and production environments.

Question 25

A data engineering team is implementing a CI/CD pipeline for deploying Declarative Automation Bundles.

Before deploying changes to production, the engineering team wants the CI/CD workflow to verify the bundle configuration and fail the deployment process if configuration issues are detected.

Which Databricks CLI command best meets these requirements?

- A. databricks bundle validate
- B. databricks bundle verify
- C. databricks bundle deploy
- D. databricks bundle run

Correct answer: A

A. databricks bundle validate

Explanation

The Databricks CLI supports Declarative Automation Bundle workflows through commands such as:

- bundle validate
- bundle deploy
- bundle run

A common CI/CD workflow is:

- Verify the bundle configuration
- Detect configuration issues early
- Deploy the bundle after validation succeeds

The bundle validate command helps identify issues such as:

- invalid configuration values
- missing deployment targets

- incorrect resource definitions
- environment configuration problems

Using validation before deployment improves:

- deployment reliability
- CI/CD automation quality
- operational safety
- deployment consistency

This approach is commonly used in automated deployment pipelines for Lakeflow Jobs, SDP pipelines, and other Databricks workspace assets.

Question 26

A Delta table receives frequent micro-batch writes from a streaming pipeline.

Over time, the table accumulates a large number of very small Parquet files, and query performance begins to degrade due to excessive file scanning overhead.

Which command should the data engineer use to improve query performance by compacting small files into larger ones?

A. MERGE INTO

B. ANALYZE TABLE

C. VACUUM

D. OPTIMIZE

Correct answer: D

D. OPTIMIZE

Explanation

Streaming pipelines that write frequent small batches to Delta tables can create many small Parquet files over time.

This small file problem can lead to:

- excessive file scanning
- increased metadata overhead
- slower query performance

The OPTIMIZE command rewrites small files into a smaller number of larger files, improving read efficiency and query performance.

OPTIMIZE is commonly used as part of Delta Lake maintenance and performance optimization workflows.

Question 27

A data engineering team is building a Lakeflow Job with the following workflow:

- execute an existing Lakeflow Spark Declarative Pipelines (SDP) pipeline that builds Bronze and Silver tables
- execute a notebook that performs a post-processing step after the pipeline completes

The engineering team wants to use the most appropriate Lakeflow Job task types for each workload.

Which approach best meets these requirements?

- A.** Use a SQL task for the SDP pipeline and a notebook task for the post-processing notebook.
- B.** Use a Run Job task for the SDP pipeline and a notebook task for the post-processing notebook.
- C.** Use a pipeline task for the SDP pipeline and a notebook task for the post-processing notebook.
- D.** Use a notebook task for the SDP pipeline and a notebook task for the post-processing notebook.

Correct answer: C

- C.** Use a pipeline task for the SDP pipeline and a notebook task for the post-processing notebook.

Explanation

Lakeflow Jobs supports multiple task types designed for different orchestration scenarios.

Common task types include:

- Pipeline tasks for Lakeflow Spark Declarative Pipelines workloads
- Notebook tasks for notebook-based processing
- SQL tasks for SQL query execution
- Run Job tasks for orchestrating other jobs

Using the appropriate task type improves:

- orchestration clarity
- operational monitoring

- workflow maintainability
- task-level visibility

Pipeline tasks are the preferred approach when directly orchestrating Lakeflow Spark Declarative Pipelines workloads within a Lakeflow Job.

Question 28

A data engineering team grants the following privileges to a group named `data_analysts`:

```
GRANT USE CATALOG ON CATALOG finance TO data_analysts;
```

```
GRANT USE SCHEMA ON SCHEMA finance.reporting TO data_analysts;
```

```
GRANT SELECT ON TABLE finance.reporting.customer_sales TO data_analysts;
```

Later, the engineering team runs the following command:

```
REVOKE SELECT ON TABLE finance.reporting.customer_sales FROM data_analysts;
```

After the revoke operation, members of the `data_analysts` group can still access the table successfully.

Which situation most likely explains this behavior?

- A. The `USE SCHEMA` privilege automatically includes `SELECT` access on all tables in the schema.
- B. `SELECT` privileges cannot be revoked from Delta tables.
- C. The users belong to another group that also has the `SELECT` privilege on the table.
- D. `REVOKE` only removes privileges after the cluster is restarted.

Correct answer: C

C. The users belong to another group that also has the `SELECT` privilege on the table.

Explanation

In Unity Catalog, users can receive effective permissions through multiple sources, including:

- direct user grants
- multiple group memberships
- inherited privilege assignments

As a result, revoking a privilege from one group does not necessarily remove access if another grant path still provides the same permission.

In this scenario:

- the engineering team revoked `SELECT` from the `data_analysts` group
- however, the users still likely receive `SELECT` access through another group membership

This is a common operational troubleshooting scenario in environments where users belong to multiple governance groups.

Question 29

A data engineer wants to create a SQL user-defined function (UDF) in Databricks that:

- accepts a first name and last name
- capitalizes each value
- returns the formatted full name

Which statement correctly creates this function?

A.

```
CREATE FUNCTION format_name(fname STRING, lname STRING)
RETURNS STRING
AS 'SELECT UPPER(fname) || " " || UPPER(lname)';
```

B.

```
CREATE FUNCTION format_name(fname STRING, lname STRING)
RETURNS STRING
USING 'INITCAP(fname) || INITCAP(lname)';
```

C.

```
CREATE OR REPLACE FUNCTION format_name(fname, lname)
RETURN INITCAP(fname) || ' ' || INITCAP(lname);
```

D.

```
CREATE OR REPLACE FUNCTION format_name(fname STRING, lname STRING)
RETURNS STRING
RETURN INITCAP(fname) || ' ' || INITCAP(lname);
```

Correct answer: D

Explanation

In Databricks SQL, SQL user-defined functions (UDFs) are created using `CREATE FUNCTION` or `CREATE OR REPLACE FUNCTION`. SQL UDFs require:

- parameter names and data types
- a return type
- a valid SQL expression using the `RETURN` keyword

For example:

```
CREATE OR REPLACE FUNCTION format_name(fname STRING, lname STRING)
RETURNS STRING
RETURN INITCAP(fname) || ' ' || INITCAP(lname);
```

This function:

- accepts two string inputs
- capitalizes the first letter of each value using INITCAP
- concatenates the values using ||
- returns the formatted full name

Question 30

A retail company currently maintains separate platforms for BI reporting, real-time analytics, and machine learning workloads, leading to:

- inconsistent business metrics
- duplicated storage costs
- separate governance processes across platforms

Leadership wants to modernize the platform while continuing to store data in cloud object storage.

Which approach best meets these requirements?

- A.** Implement a Lakehouse architecture using Delta Lake tables
- B.** Maintain separate analytics and machine learning platforms while synchronizing data between systems using scheduled ETL pipelines
- C.** Implement a cloud-based data platform using Parquet tables in object storage with separate processing and governance workflows for different teams
- D.** Use a traditional enterprise data warehouse for reporting and a separate streaming platform for real-time analytics

Correct answer: A

A. Implement a Lakehouse architecture using Delta Lake tables

Explanation

The company is trying to modernize its analytics platform while reducing the operational complexity caused by maintaining separate systems for reporting, streaming analytics, and machine learning workloads.

A Lakehouse architecture allows organizations to use cloud object storage as the foundation for multiple workload types while adding capabilities such as:

- transactional reliability
- centralized governance
- support for both streaming and batch processing
- reduced data duplication across teams

This enables organizations to work from a more consistent and governed source of data without maintaining separate copies across multiple platforms.

Question 31

A manufacturing company ingests IoT telemetry events into a Bronze table using PySpark.

Due to intermittent network retries, some telemetry events are written multiple times into the Bronze layer.

Each record contains:

- machine_id
- event_timestamp
- temperature

The engineering team identifies that a telemetry event should be considered unique based on the combination of:

- machine_id
- event_timestamp

Which transformation best removes duplicate telemetry events before writing to the Silver table?

A.

```
silver_df = bronze_df.dropDuplicates(  
    ["machine_id", "event_timestamp"]  
)
```

B.

```
silver_df = bronze_df.distinct(  
    ["machine_id", "event_timestamp"]  
)
```

C. `silver_df = bronze_df.dropDuplicates()`

D.

```
silver_df = bronze_df.drop(  
    "machine_id",  
    "event_timestamp"  
)
```

Correct answer: A

Explanation

The `dropDuplicates()` function is used in PySpark to remove duplicate rows from a DataFrame.

When specific columns are provided:

```
dropDuplicates(["machine_id", "event_timestamp"])
```

PySpark uses the combination of those columns to identify duplicate records.

For example, if the Bronze table contains:

machine_id	event_timestamp	temperature
M101	2026-05-10 10:15:00	45.2
M101	2026-05-10 10:15:00	46.1

PySpark would treat these rows as duplicates because they share the same:

- machine_id

- event_timestamp

After applying `dropDuplicates(["machine_id", "event_timestamp"])`, one of the rows would be retained and the other removed.

The specific row retained is not guaranteed unless additional ordering logic is applied before deduplication.

Question 32

A global company uses multiple Databricks workspaces across different business units.

The company currently manages data access separately within each workspace, resulting in:

- inconsistent governance policies
- duplicated permission management
- limited visibility into data lineage across teams

Leadership wants a centralized governance solution while continuing to store data in cloud object storage.

Which Databricks component best addresses these requirements?

A. Delta Live Tables

B. Delta Lake

C. Unity Catalog

D. Hive metastore

Correct answer: C

C. Unity Catalog

Explanation

The company's main challenge is fragmented governance across multiple Databricks workspaces.

Unity Catalog provides a centralized governance layer that allows organizations to:

- manage permissions consistently
- track lineage across workloads
- audit data access centrally
- govern data across multiple workspaces

This helps organizations standardize governance while continuing to store data in cloud object storage as part of a Lakehouse architecture.

Question 33

A data engineering team is investigating a slow Spark workload in Databricks.

While reviewing the Spark UI, the engineering team notices:

- high spill-to-disk metrics across multiple tasks
- heavy disk I/O during shuffle operations
- significantly slower stage execution compared to previous runs

Which issue is the most likely cause of this behavior?

- A.** Data skew causing uneven partition distribution across executors.
- B.** Excessive shuffle partition creation causing Spark to spend more time coordinating shuffle tasks.
- C.** Liquid Clustering reorganizing Delta files during query execution.
- D.** Disk spilling caused by insufficient executor memory during shuffle-heavy processing.

Correct answer: D

D. Disk spilling caused by insufficient executor memory during shuffle-heavy processing.

Explanation

Disk spilling is a common Spark performance bottleneck that occurs when Spark does not have enough memory to process intermediate shuffle data in memory.

Spilling commonly occurs during:

- joins
- aggregations
- sorts
- other shuffle-heavy transformations

Typical Spark UI symptoms include:

- high spill-to-disk metrics

- increased disk I/O
- slower stage execution
- shuffle-related performance degradation

When spilling occurs, Spark writes intermediate data to disk instead of processing it entirely in memory, which significantly reduces performance.

The Spark UI helps engineering teams diagnose spilling-related bottlenecks through stage-level execution metrics and spill statistics.

Question 34

A retail company receives CSV sales files daily in an ADLS container. The engineering team wants to:

- incrementally ingest only new files
- avoid reprocessing previously loaded files
- load the data into Unity Catalog governed Delta tables
- use a SQL-based ingestion approach

Which solution best meets these requirements?

- A.** Use an external table
- B.** Use CREATE TABLE AS SELECT
- C.** Use INSERT INTO with overwrite mode
- D.** Use COPY INTO

Correct answer: D

D. Use COPY INTO

Explanation

The requirement is to ingest newly arriving CSV files from cloud object storage into a governed Delta table using SQL while avoiding duplicate file processing.

COPY INTO is the best fit because it is designed for incremental file ingestion from cloud object storage into Delta tables. It tracks files that have already been loaded and processes only new files during later runs.

Question 35

A data engineering team is building a Lakeflow Job to process Formula 1 race result files uploaded to a Unity Catalog Volume.

New race result files are uploaded after a race completes during race weekends. Some weekends contain races, while other weekends do not. Occasionally, sprint race and main race result files may arrive multiple times during the same weekend.

The engineering team wants:

- the job to start when new files arrive
- avoid unnecessary job runs when no new files are available
- minimize unnecessary compute usage

Which approach best meets these requirements?

- A.** Configure the Lakeflow Job using a file arrival trigger on the Unity Catalog Volume.
- B.** Configure the Lakeflow Job using a scheduled weekly trigger and a dedicated task that checks whether new files exist before continuing the job.
- C.** Configure the Lakeflow Job using a scheduled weekly trigger and check at the beginning of every notebook whether new files exist before processing.
- D.** Configure the Lakeflow Job to run on demand only through manual execution after each race weekend.

Correct answer: A

- A.** Configure the Lakeflow Job using a file arrival trigger on the Unity Catalog Volume.

Explanation

Lakeflow Jobs supports multiple trigger types, including:

- scheduled triggers
- file arrival triggers
- table update triggers

File arrival triggers are designed for workloads where data arrives unpredictably rather than on a fixed schedule.

In this scenario:

- race weekends do not occur every weekend
- files may arrive multiple times during some weekends
- no files may arrive for extended periods

Using a scheduled trigger would result in unnecessary job executions when no new data exists.

A file arrival trigger allows the Lakeflow Job to start automatically only when new files are detected in the Unity Catalog Volume, improving:

- orchestration efficiency
- compute utilization
- operational automation

Question 36

A company wants to ingest ServiceNow data into Unity Catalog governed Delta tables with:

- minimal custom coding
- managed ingestion infrastructure
- centralized governance
- reliable scheduling support

Which ingestion approach best meets these requirements?

- A.** Use Auto Loader for SaaS ingestion
- B.** Use Lakeflow Connect managed SaaS connectors
- C.** Use REST API based ingestion
- D.** Use COPY INTO from the SaaS application

Correct answer: B

- B.** Use Lakeflow Connect managed SaaS connectors

Explanation

The scenario focuses on ingesting enterprise SaaS application data into Databricks with minimal operational overhead and centralized governance support.

Lakeflow Connect managed SaaS connectors are designed to:

- simplify ingestion from supported SaaS applications
- reduce custom ingestion code
- provide managed ingestion infrastructure
- support centralized governance with Unity Catalog
- simplify operational management and scheduling

This makes them well suited for enterprise SaaS ingestion workloads.

Question 37

A data engineering team wants to reduce the manual operational effort required to maintain performant Unity Catalog managed Delta tables.

The engineering team wants optimization operations such as:

- file compaction
- statistics collection
- cleanup of unused data files

to run automatically.

Which Databricks feature best meets these requirements?

- A. Predictive Optimization**
- B. Adaptive Query Execution
- C. OPTIMIZE
- D. Liquid Clustering

Correct answer: A

A. Predictive Optimization

Explanation

Predictive Optimization is a Databricks feature that automatically performs maintenance and optimization operations for Unity Catalog managed Delta tables.

Predictive Optimization can automatically run operations such as:

- OPTIMIZE
- VACUUM
- ANALYZE

This helps engineering teams:

- reduce manual maintenance effort
- improve query performance
- maintain efficient Delta table layouts
- automate operational optimization tasks

Liquid Clustering is a separate feature that improves clustering flexibility and data organization for Delta tables but does not replace Predictive Optimization maintenance operations.

Question 38

A data engineering team currently runs nightly ETL pipelines on shared all-purpose compute used by analysts and engineers throughout the day.

Over time, the team has observed:

- resource contention with interactive notebook users
- increased compute costs from idle resources
- the ETL pipelines requiring specific runtime version and library setup for each scheduled run

Which approach would best address these issues around running ETL pipelines?

- A.** Increase the size of the all-purpose compute
- B.** Use job compute for the scheduled ETL pipelines
- C.** Run the ETL pipelines on single-node compute
- D.** Use serverless compute for all workloads

Correct answer: B

- B.** Use job compute for the scheduled ETL pipelines

Explanation

The main issue in this scenario is that production ETL pipelines are sharing compute resources with interactive notebook users. This creates operational challenges such as resource contention and unnecessary idle compute costs.

In addition, the pipelines require consistent runtime versions and library configurations for reliable scheduled execution. Job compute is designed for scheduled and automated workloads and provisions dedicated compute environments specifically for each job run. This provides better workload isolation, more predictable execution environments, and improved cost efficiency for production ETL pipelines.

Question 39

A healthcare organization ingests semi-structured JSON records using Auto Loader.

Over time:

- new fields are added to incoming records
- some existing fields occasionally stop appearing in incoming records
- nested structures occasionally change
- ingestion pipelines should continue processing without frequent manual schema updates

Which Auto Loader capability best supports these requirements?

- A.** Schema drift
- B.** Schema inference
- C.** Schema evolution
- D.** mergeSchema

Correct answer: C

C. Schema evolution

Explanation

The scenario focuses on continuously ingesting semi-structured JSON data where source schemas evolve over time.

Auto Loader schema evolution capabilities help ingestion pipelines:

- adapt to newly added fields
- continue processing evolving nested structures
- reduce manual schema maintenance
- support long-running ingestion pipelines despite changing source schemas

This is especially important for large-scale semi-structured ingestion workloads where source systems evolve independently over time.

Question 40

A retail company ingests daily product and pricing files into a Bronze layer Delta table using cloud object storage as the landing zone. The ingestion process is fully automated and runs several times throughout the day as new files arrive from multiple regional systems.

During a recent auditing investigation, the data engineering team was asked to identify from the Bronze table:

- which source file produced specific records
- when individual records were ingested
- which regional system produced specific records

The team wants to improve ingestion traceability and support future operational investigations without redesigning the overall ingestion architecture.

Which implementation best supports these requirements?

- A.** Create a separate Bronze lookup table containing file path, ingestion timestamp, and region information during ingestion
- B.** Enable Delta table history tracking on the Bronze table to capture file path, ingestion timestamp, and region information
- C.** Enable Lakeflow Jobs to capture file path, ingestion timestamp, and region information
- D.** Capture source metadata columns such as file path, ingestion timestamp, and region during Bronze table ingestion

Correct answer: D

D. Capture source metadata columns such as file path, ingestion timestamp, and region during Bronze table ingestion

Explanation

The scenario focuses on operational auditing and ingestion traceability within Bronze layer ingestion pipelines.

Capturing source metadata directly during ingestion helps organizations:

- trace records back to original source files
- identify ingestion timing

- track originating systems or regions
- support auditing and compliance investigations
- improve operational troubleshooting and lineage visibility

Common metadata captured during ingestion includes:

- source file path
- ingestion timestamp
- batch identifiers
- source system or region identifiers

This metadata is commonly stored directly alongside Bronze records to simplify downstream investigations and operational support.

Question 41

A data engineering team is investigating a slow Spark workload in Databricks.

While reviewing the Spark UI, the engineering team notices that:

- one task in a stage is taking significantly longer than all other tasks
- most executors finish quickly while one executor continues processing for an extended period
- shuffle read sizes are highly uneven across tasks

Which issue is the most likely cause of this behavior?

- A.** Data skew causing uneven partition distribution across executors.
- B.** Disk spilling caused by insufficient executor memory.
- C.** Predictive optimization creating too many small files during execution.
- D.** Excessive cluster startup time delaying task scheduling.

Correct answer: A

- A.** Data skew causing uneven partition distribution across executors.

Explanation

Data skew is a common Spark performance bottleneck where data is distributed unevenly across partitions.

Common symptoms visible in the Spark UI include:

- one or a few tasks running much longer than others
- uneven shuffle read sizes
- executors finishing at very different times
- long-running stages despite most tasks completing quickly

Data skew commonly occurs during:

- joins

- aggregations
- shuffle-heavy transformations

Skew can lead to:

- poor parallelism
- long-running stages
- inefficient resource utilization
- extended pipeline runtimes

The Spark UI helps engineering teams identify skew-related bottlenecks through stage-level execution metrics and task distribution analysis.

Question 42

A data engineer adds new environment variables to a Databricks cluster using init scripts to support connectivity to an external storage system.

After saving the changes and running a notebook, the new environment variables are not available.

What should the engineer do to ensure the updated environment variables are applied?

- A. Re-run all notebook cells from the beginning
- B. Detach and reattach the notebook to the cluster.
- C. Create a new notebook session.
- D. Restart the cluster

Correct answer: D

D. Restart the cluster

Explanation

In Databricks, changes to cluster-level configuration such as:

- environment variables
- init scripts
- installed libraries

are applied during cluster startup. After modifying these settings, the cluster must be restarted so the runtime environment is recreated with the updated configuration.

Question 43

A data engineering team is building a Lakeflow Job with the following tasks:

- ingest_bronze_data
- validate_silver_data
- build_gold_reports
- send_failure_notification

The engineering team wants:

- build_gold_reports to execute if the validation task succeeds
- send_failure_notification to execute if the validation task fails

Which approach best meets these requirements?

- A.** Configure a **Case when task** after validate_silver_data to control downstream branching.
- B.** Configure sequential notebook tasks and use notebook-level if statements to control downstream branching.
- C.** Configure a **Switch task** after validate_silver_data to control downstream branching.
- D.** Configure an **If/else task** after validate_silver_data to control downstream branching.

Correct answer: D

- D.** Configure an If/else task after validate_silver_data to control downstream branching.

Explanation

Lakeflow Jobs supports conditional workflow execution using If/else tasks.

Conditional branching is commonly used when workflows require different downstream behavior depending on:

- validation outcomes
- quality checks

- task success or failure
- business rule evaluation

In this scenario:

- Gold reporting should execute only after successful validation
- failure notifications should execute only when validation fails

Using native If/else tasks improves:

- workflow readability
- orchestration maintainability
- task-level observability
- operational reliability

Compared to notebook-based orchestration approaches, native task branching provides clearer DAG-based workflow management and better operational visibility.

Question 44

A security-conscious enterprise is evaluating Databricks and wants to ensure that all customer data, including file reads/writes and notebook execution, remains within their own cloud environment. However, they are okay with Databricks managing the platform services like the web UI and job scheduler.

Which aspect of the Databricks architecture makes this setup possible?

- A. Jobs run on ephemeral clusters with encrypted secrets stored in the control plane.
- B. Unity Catalog separates secure and insecure clusters using data tags.
- C. Databricks SQL provides serverless access to files across multiple regions.
- D. The control plane is hosted by Databricks, while the data plane runs in the customer's cloud account.

Correct answer: D

- D. The control plane is hosted by Databricks, while the data plane runs in the customer's cloud account.

Explanation

Databricks uses a control plane and data plane architecture where Databricks manages platform services such as the web UI, job scheduling, and APIs in the control plane, while customer data processing, notebook execution, and file access occur within the customer's cloud environment in the data plane.

Question 45

A data engineer is building a Spark Structured Streaming pipeline to process IoT sensor data.

The engineering team wants the pipeline to:

- process new data every 60 seconds
- use a micro-batch processing model
- continue running continuously as new data arrives

Which trigger option should the data engineer use for the trigger configuration?

- A. `Trigger.Continuous("60 seconds")`
- B. `Trigger.AvailableNow()`
- C. `Trigger.ProcessingTime("1 minute")`
- D. `Trigger.Once()`

Correct answer: C

C. `Trigger.ProcessingTime("1 minute")`

Explanation

In Spark Structured Streaming, triggers control how frequently streaming queries process incoming data.

When using a micro-batch processing model, `Trigger.ProcessingTime()` is commonly used to define recurring execution intervals.

For example:

```
Trigger.ProcessingTime("1 minute")
```

configures Spark to:

- check for new data every 60 seconds
- process available records as a micro-batch
- continue running continuously

This model supports common streaming operations such as:

- aggregations
- joins
- watermarking
- checkpointing

`Trigger.Continuous()` uses continuous processing mode instead of standard micro-batching and is not typically used for scheduled micro-batch processing workloads.

`Trigger.Once()` and `Trigger.AvailableNow()` process data and then terminate rather than running continuously.

Question 46

A company uses Lakeflow Spark Declarative Pipelines to process IoT sensor data from manufacturing equipment.

New data arrives in cloud storage approximately every 3 to 4 hours. The engineering team needs to process this data reliably but does not require near real time insights. They also want to minimize compute costs by avoiding unnecessary compute usage between pipeline runs.

Which pipeline mode best suits this use case?

- A. Triggered pipeline mode
- B. Streaming Auto Loader mode
- C. Continuous pipeline mode
- D. Interactive pipeline mode

Correct answer: A

A. Triggered pipeline mode

Explanation

Lakeflow Spark Declarative Pipelines supports two primary execution modes:

- Triggered pipeline mode
- Continuous pipeline mode

Triggered mode is designed for workloads where data arrives periodically and does not require continuous low-latency processing. The pipeline processes available data during a run and then stops automatically after completion.

Continuous mode keeps compute resources running continuously to process new data with lower latency, making it better suited for near real time streaming workloads.

Since this workload receives new data only every few hours and prioritizes cost efficiency over low-latency processing, Triggered pipeline mode is the most appropriate choice.

Question 47

A data engineering team notices that a Lakeflow Job that previously completed in 18 minutes is now consistently taking more than 40 minutes to finish.

The engineering team wants to identify when the performance degradation started by comparing recent executions against historical job runs.

Which approach best meets this requirement?

- A. Export the Lakeflow Job as a DBC file and compare it with previous exports.
- B. Use the Spark UI to compare the performance of the Lakeflow Job.
- C. Use the Lakeflow Jobs Runs tab to compare current and historical job execution metrics.
- D. Use the event history of the job compute to compare cluster performance over time.

Correct answer: C

C. Use the Lakeflow Jobs Runs tab to compare current and historical job execution metrics.

Explanation

The Lakeflow Jobs monitoring UI provides operational visibility into job execution history and pipeline health.

The Runs tab helps engineering teams:

- compare current runtimes against historical baselines
- identify performance degradation trends
- investigate failed or slow runs
- review task execution details over time

This helps teams diagnose issues such as:

- increasing data volumes
- inefficient transformations
- resource bottlenecks
- infrastructure-related slowdowns

Monitoring historical execution trends is an important operational practice for maintaining reliable and performant data pipelines.

Question 48

A data engineering team is building a Lakeflow Job with the following tasks:

- ingest_customers
- ingest_orders
- build_silver_customers
- build_silver_orders
- build_gold_sales

The engineering team wants:

- ingest_customers and ingest_orders to run in parallel
- each Silver task to begin only after its corresponding ingestion task completes successfully
- build_gold_sales to execute only after both Silver tasks complete successfully

Which approach best meets these requirements?

A. Configure all tasks sequentially in the following order:

ingest_customers → ingest_orders → build_silver_customers → build_silver_orders → build_gold_sales

B. Configure build_silver_customers to depend on ingest_customers, configure build_silver_orders to depend on ingest_orders, and configure build_gold_sales to depend on both Silver tasks.

C. Configure both Silver tasks to depend on both ingestion tasks, then configure build_gold_sales to depend on both Silver tasks.

D. Create a master notebook that runs the ingestion and transformation notebooks sequentially using %run commands, then run the master notebook using a lakeflow job.

Correct answer: B

B. Configure build_silver_customers to depend on ingest_customers, configure build_silver_orders to depend on ingest_orders, and configure build_gold_sales to depend on both Silver tasks.

Explanation

Lakeflow Jobs uses a DAG-based task graph to manage workflow orchestration.

A well-designed DAG should:

- maximize parallel execution where possible
- avoid unnecessary task dependencies
- clearly define upstream and downstream relationships

In this scenario:

- ingestion tasks can run independently in parallel
- each Silver task depends only on its corresponding ingestion task
- the Gold task depends on both Silver tasks because it requires outputs from both pipelines

This dependency structure improves:

- execution efficiency
- pipeline maintainability
- observability
- retry handling

Compared to older notebook orchestration approaches using %run, Lakeflow Jobs provides native dependency management, task-level monitoring, retries, and scalable orchestration capabilities.

Question 49

A data engineering team is building a Silver customer table using Lakeflow Spark Declarative Pipelines.

The Bronze customer table contains some records where `customer_id` is `NULL`. These records should be treated as data quality issues and dropped from the Silver table, while still being tracked by the pipeline's data quality metrics.

Which code snippet best meets this requirement?

A.

```
@dp.table
@dp.expect("valid_customer_id", "customer_id IS NOT NULL")
def silver_customers():
    return spark.read.table("formula1.bronze.customers")
```

B.

```
@dp.table
@dp.expect_or_drop("valid_customer_id", "customer_id IS NOT NULL")
def silver_customers():
    return spark.read.table("formula1.bronze.customers")
```

C.

```
@dp.table
@dp.expect_or_fail("valid_customer_id", "customer_id IS NOT NULL")
def silver_customers():
    return spark.read.table("formula1.bronze.customers")
```

D.

```
@dp.table
@dp.expect_or_drop("valid_customer_id", "customer_id IS NULL")
def silver_customers():
    return spark.read.table("formula1.bronze.customers")
```

Correct answer: B

Explanation

Lakeflow Spark Declarative Pipelines supports declarative data quality expectations that can:

- track invalid records
- drop invalid records
- fail pipeline updates when quality checks fail

In this scenario, the requirement is to:

- prevent invalid records from reaching the trusted Silver layer
- continue processing valid records

- maintain visibility into data quality issues through pipeline metrics

`@dp.expect_or_drop()` is specifically designed for this pattern.

Question 50

A company wants to store data in cloud object storage while still supporting:

- transactional reliability
- schema enforcement
- support for both batch and streaming workloads
- the ability to access historical versions of data

Which technology provides these capabilities?

- A. Delta Lake**
- B. Delta Live Tables
- C. Parquet
- D. Unity Catalog

Correct answer: A

A. Delta Lake

Explanation

The company wants to continue using scalable cloud object storage while adding capabilities commonly required for reliable data engineering and analytics workloads.

Delta Lake extends cloud object storage with features such as:

- ACID transactions
- schema enforcement
- support for both batch and streaming processing
- historical table versioning through Time Travel

These capabilities allow organizations to build reliable Lakehouse architectures while continuing to use open storage formats and cloud object storage.

Parquet is only a file format, while Delta Lake builds additional reliability and table management capabilities on top of storage formats such as Parquet.

Question 51

A data engineering team is building a Lakeflow Job that loads daily supplier inventory files into a Bronze table.

The ingestion task occasionally fails because the supplier's cloud storage service temporarily throttles requests during peak hours. In most cases, rerunning the task a few minutes later succeeds successfully.

The engineering team wants the job to automatically recover from these temporary failures without requiring manual reruns of the job.

Which approach best meets these requirements?

- A.** Increase the auto-scaling limits for the job cluster.
- B.** Convert the ingestion process into a continuously running Lakeflow Spark Declarative Pipelines pipeline.
- C.** Create a master notebook that reruns failed notebooks sequentially using %run commands.
- D.** Configure retries and retry intervals on the ingestion task.

Correct answer: D

D. Configure retries and retry intervals on the ingestion task.

Explanation

Lakeflow Jobs supports configurable retry behavior at the task level.

Retries are commonly used for workloads that interact with external systems that may occasionally experience:

- temporary throttling
- intermittent network failures
- transient cloud storage issues
- short-lived service interruptions

By configuring:

- retry count
- retry interval

Lakeflow Jobs can automatically rerun failed tasks without requiring operators to manually restart the job.

This improves:

- operational resilience
- workflow reliability
- automated recovery from transient failures

Task-level retries are commonly used for ingestion workloads where temporary external failures are expected occasionally.

Question 52

A data engineering team is developing a new Lakeflow Spark Declarative Pipelines enhancement inside a Databricks Git Folder connected to a shared Git repository.

The engineer responsible for the enhancement wants to:

- experiment with pipeline changes safely
- avoid exposing unfinished code to the rest of the team
- continue collaborating through the shared repository without exposing unfinished changes

Which approach best meets these requirements?

- A.** Create a second copy of the project inside the workspace and manually merge notebook changes later.
- B.** Create and switch to a feature branch inside the Databricks Git Folder before continuing development.
- C.** Disconnect the Databricks Git Folder from the remote repository while developing the enhancement.
- D.** Continue developing directly on the shared main branch and commit changes incrementally.

Correct answer: B

- B.** Create and switch to a feature branch inside the Databricks Git Folder before continuing development.

Explanation

Databricks Git Folders support Git-based collaborative development directly within the Databricks workspace UI.

Feature branches are commonly used to:

- isolate development work
- safely test new features
- avoid impacting shared production-ready code
- support collaborative review workflows

A typical workflow is:

- Create a feature branch
- Make and test changes
- Commit and push changes
- Create a pull request
- Merge into the main branch after review

Using feature branches improves:

- development safety
- collaboration
- version control
- CI/CD integration

Question 53

A data engineering team is building a Gold analytics dataset using Lakeflow Spark Declarative Pipelines.

The analytics team requires:

- precomputed revenue aggregations by product category
- improved dashboard query performance
- automatic refreshes when upstream Silver tables change
- integration with Unity Catalog governance

Which dataset type best meets these requirements?

- A.** View
- B.** Materialized view
- C.** Global temporary view
- D.** Streaming table

Correct answer: B

B. Materialized view

Explanation

Materialized views are designed for analytical serving workloads where query performance is important.

Unlike standard views, materialized views physically store the results of the query definition. Lakeflow Spark Declarative Pipelines automatically refreshes the materialized view when upstream datasets change.

This helps:

- improve dashboard query performance
- reduce repeated computation
- simplify analyst access to curated Gold-layer datasets

Question 54

A data engineering team is monitoring a Lakeflow Job that contains multiple dependent tasks.

The engineering team notices that the job is still running and only some of the output tables are refreshed.

The team wants to identify whether an upstream task failure or delay is blocking downstream execution.

Which approach best meets this requirement?

- A.** Use the Spark UI to identify which Spark stages are preventing downstream Lakeflow Job tasks from starting.
- B.** Use the Lakeflow Jobs DAG view to inspect task dependencies and identify upstream blockers affecting downstream tasks.
- C.** Use the cluster metrics dashboard to determine whether cluster resource utilization is delaying table refreshes.
- D.** Use the Lakeflow Jobs Runs tab to compare historical task execution durations across previous job runs.

Correct answer: B

B. Use the Lakeflow Jobs DAG view to inspect task dependencies and identify upstream blockers affecting downstream tasks.

Explanation

The Lakeflow Jobs UI provides DAG-based workflow visualization for monitoring orchestration behavior.

The DAG view helps engineering teams:

- visualize task dependencies
- identify upstream blockers
- monitor task execution states
- investigate failed or delayed tasks
- troubleshoot orchestration issues

When only some downstream tables are refreshed, the DAG view helps determine whether:

- upstream tasks are still running
- failures occurred
- dependency conditions were not met

This operational visibility is important when troubleshooting complex Lakeflow Job workflows with multiple dependent tasks.

Question 55

A business intelligence team frequently leaves SQL warehouses running after dashboard usage ends for the day, resulting in unnecessary compute costs during idle periods.

Which configuration change would most directly help reduce these costs?

- A. Enable Auto Stop
- B. Enable Auto Scaling
- C. Enable Auto Termination
- D. Enable Photon

Correct answer: A

A. Enable Auto Stop

Explanation

The problem in this scenario is unnecessary cost caused by SQL warehouses remaining active even when no dashboard queries are running.

Auto Stop directly addresses this issue by automatically stopping SQL warehouses after inactivity is detected. This helps organizations reduce idle compute costs while still allowing warehouses to restart automatically when new queries arrive.

The question also tests the distinction between SQL warehouse terminology and cluster-based compute terminology. SQL warehouses use Auto Stop, while cluster-based compute uses Auto Termination.